Reviewer Pack — Dehn Function Exponentiation Bounds Resolution Length for Ts…

Entry 4d96b3fd6533 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 04:46:20 UTC

Dehn Function Exponentiation Bounds Resolution Length for Tseitin Formulas

Entry ID: `4d96b3fd6533` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 04:46:20 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory **Field B** (complexity object): Resolution Proof Length

Statement:

> For a Tseitin formula Φ(G) derived from a d-regular graph G on n vertices, let δ(G) denote the minimal Dehn function of a group presentation constructed from G's adjacency matrix. Then resolution length(Φ(G)) ≥ 2^{Ω(δ(G))}. For non-expanders, δ(G) ≤ poly(n) and resolution length(Φ(G)) = poly(n).

Rationale (proposer's reasoning):

> Dehn functions measure the complexity of filling loops in group presentations, mirroring the combinatorial complexity of resolution proofs. By linking group-theoretic filling properties to proof complexity, we expose structural parallels between geometric group theory and proof systems. Expanders, with their high Dehn functions, would require exponentially longer proofs, while non-expanders have polynomial bounds.

Taxonomy category: `GCT` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `995945046c796e66`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_d_regular_graph(n, d):
    if (d * n) % 2 != 0:
        return None
    adj_matrix = [[0] * n for _ in range(n)]
    edges = set()
    while len(edges) < d * n // 2:
        u, v = random.sample(range(n), 2)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            adj_matrix[u][v] = 1
            adj_matrix[v][u] = 1
            edges.add((u, v))
    return adj_matrix

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below pivot
        factor = 1 / A[i][i]
        for j in range(i, n):
            A[i][j] *= factor
        for k in range(i+1, n):
            factor = A[k][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
    return A
```

```
def dehn_function(G):
    n = len(G)
    I = [[int(i == j) for j in range(n)] for i in range(n)]
    A = gaussian_elimination(G + I)
    delta_G = 0
    for i in range(n):
        if A[i][i] != 1:
            return float('inf')
        delta_G += sum(abs(A[j][i]) for j in range(i+1, n))
    return delta_G

def resolution_length(phi):
    stack = phi[:]
    length = 0
    while stack:
        clause = stack.pop()
        if len(clause) == 1:
            continue
        literal = random.choice(clause)
        new_clauses = []
        for c in stack:
            if literal not in c and -literal not in c:
                new_clauses.append([l for l in c if l != -literal])
        stack.extend(new_clauses)
        length += 1
    return length

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    d = 3
    phi = []
    G = generate_d_regular_graph(n, d)
    if G is None:
        return {
            "metric_name": "resolution_length",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "graph_not_d_regular"
        }

    delta_G = dehn_function(G)
    if delta_G == float('inf'):
        return {
            "metric_name": "resolution_length",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "dehn_function_infinite"
        }

    length = resolution_length(phi)
    conjecture_holds = length >= 2 ** (0.5 * delta_G)
    counterexample = "" if conjecture_holds else f"resolution_length={length}, dehn_function={delt
```

```
    return {
        "metric_name": "resolution_length",
        "metric_value": length,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 3 for i in range(5, 8

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_length = sum(r["metric_value"] for r in results if r["conjecture_holds"])
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_length/len(results)} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_length/len(results)} std=0.0 support_fraction={suppo
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        counterexample = results[seeds.index(first_failing_seed)]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb7beb6.py", line 120, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb7beb6.py", line 92, in run_trial
    delta_G = dehn_function(G)
              ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb7beb6.py", line 53, in dehn_function
    A = gaussian_elimination(G + I)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb7beb6.py", line 43, in gaussian_elim
    A[i][j] *= factor
    ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

Test crashed during Gaussian elimination with index error, preventing data collection. Pre-registered support condition cannot be evaluated. | next: Fix matrix indexing in Gaussian elimination and revalidate dehn_function implementation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	100540
2	preregistration	ollama_remote	qwen3:8b	0	0	24810
3	novelty	ollama_remote	qwen3:8b	0	0	20661
4	novelty	ollama_remote	qwen3:8b	0	0	13690
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18068
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12492
7	judge	ollama_remote	qwen3:8b	0	0	17606

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 207867 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/4d96b3fd6533.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4d96b3fd6533.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4d96b3fd6533.tar.gz` (if generated)